



University
of Exeter

COMM113

Deep Learning – Week 9

Transformer and Language Models I

Gaojie (Jay) Jin
g.jin@exeter.ac.uk

Attention Is All You Need

We saw that attention dramatically improves the performance of RNNs.

Transformers take this idea one step further!

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

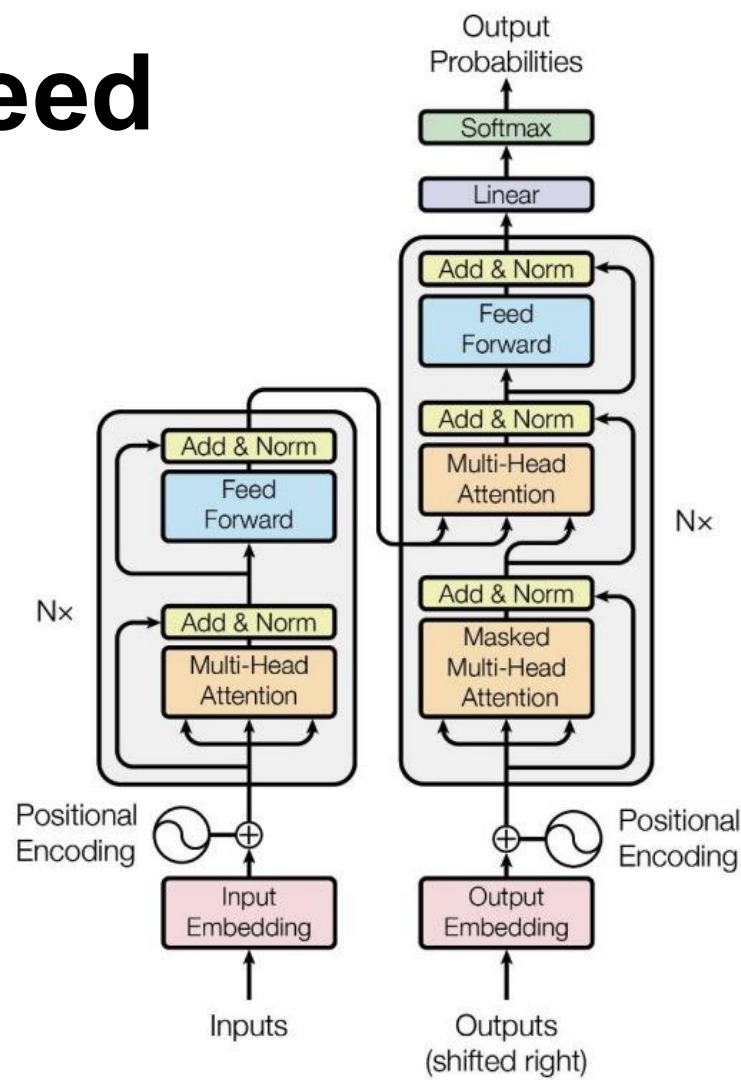
Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com



Attention Is All You Need

We saw that attention dramatically improves the performance of RNNs.

Q: What is attention?

Transformers take this idea one step further!

Attention Is All You Need

Ashish Vaswani*

Google Brain

avaswani@google.com

Noam Shazeer*

Google Brain

noam@google.com

Niki Parmar*

Google Research

nikip@google.com

Jakob Uszkoreit*

Google Research

usz@google.com

Llion Jones*

Google Research

llion@google.com

Aidan N. Gomez* †

University of Toronto

aidan@cs.toronto.edu

Łukasz Kaiser*

Google Brain

lukaszkaiser@google.com

Illia Polosukhin* †

illia.polosukhin@gmail.com

Attention Is All You Need

We saw that attention dramatically improves the performance of RNNs.

Transformers take this idea one step further!

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Q: What is attention?

A: Determines the importance of each component in a sequence

Q: Why might removing RNNs be advantageous?

Attention Is All You Need

We saw that attention dramatically improves the performance of RNNs.

Transformers take this idea one step further!

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaier@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Q: What is attention?

A: Determines the importance of each component in a sequence

Q: Why might removing RNNs be advantageous?

A: Better parallelism, fewer gradient problems, faster training.

Great Results With Transformers As Chatbots

Transformer-based models dominate the LMSYS Chatbot Arena Leaderboard!

Rank* (UB)	Model	Arena Score	95% CI	Votes	Organization	License	Knowledge Cutoff
1	ChatGPT-4o-latest (2024-08-08)	1316	+4/-3	31148	OpenAI	Proprietary	2023/10
2	Gemini-1.5-Pro-Exp-0827	1300	+4/-4	22844	Google	Proprietary	2023/11
2	Grok-2-08-13	1294	+4/-4	16215	xAI	Proprietary	2024/3
4	GPT-4o-2024-05-13	1285	+3/-2	86306	OpenAI	Proprietary	2023/10
5	GPT-4o-mini-2024-07-18	1274	+4/-4	26088	OpenAI	Proprietary	2023/10
5	Claude 3.5 Sonnet	1270	+3/-3	56674	Anthropic	Proprietary	2024/4
5	Gemini-1.5-Flash-Exp-0827	1268	+5/-4	16780	Google	Proprietary	2023/11
5	Grok-2-Mini-08-13	1267	+4/-4	16731	xAI	Proprietary	2024/3
5	Meta-Llama-3.1-405b-Instruct	1266	+4/-4	27397	Meta	Llama 3.1 Community	2023/12

<https://lmarena.ai/?leaderboard>

Transformer isn't theoretical
— it shows up in real-world
leaderboards.



Q: Why do you think human
preference is used instead of
accuracy?

Chiang et al. Chatbot Arena: An Open Platform for
Evaluating LLMs by Human Preference. 2024.

Great Results With Transformers As Chatbots

Transformer-based models dominate the LMSYS Chatbot Arena Leaderboard!

Rank* (UB)	Model	Arena Score	95% CI	Votes	Organization	License	Knowledge Cutoff
1	ChatGPT-4o-latest (2024-08-08)	1316	+4/-3	31148	OpenAI	Proprietary	2023/10
2	Gemini-1.5-Pro-Exp-0827	1300	+4/-4	22844	Google	Proprietary	2023/11
2	Grok-2-08-13	1294	+4/-4	16215	xAI	Proprietary	2024/3
4	GPT-4o-2024-05-13	1285	+3/-2	86306	OpenAI	Proprietary	2023/10
5	GPT-4o-mini-2024-07-18	1274	+4/-4	26088	OpenAI	Proprietary	2023/10
5	Claude 3.5 Sonnet	1270	+3/-3	56674	Anthropic	Proprietary	2024/4
5	Gemini-1.5-Flash-Exp-0827	1268	+5/-4	16780	Google	Proprietary	2023/11
5	Grok-2-Mini-08-13	1267	+4/-4	16731	xAI	Proprietary	2024/3
5	Meta-Llama-3.1-405b-Instruct	1266	+4/-4	27397	Meta	Llama 3.1 Community	2023/12

<https://lmarena.ai/?leaderboard>

Transformer isn't theoretical
— it shows up in real-world
leaderboards.



Chiang et al. Chatbot Arena: An Open Platform for
Evaluating LLMs by Human Preference. 2024.

Q: Why do you think human
preference is used instead of
accuracy?
A: Chatbots are judged on fluency,
coherence, helpfulness, not just
correctness.

Great Results With Transformers On NLP Tasks

Transformers outperform...!

SuperGLUE is a suite of challenging NLP tasks, including question-answering, word sense disambiguation, coreference resolution, and natural language inference.

	Rank	Name	Model	URL	Score	BoolQ	CB	COPA	MultIRC	ReCoRD	RTE	WiC	WSC	AX-b	AX-g
+	1	Inspur Cloud	Hairuo		91.4	92.5	96.5/97.6	100.0	90.5/67.9	94.1/93.2	92.8	76.1	100.0	64.6	96.1/94.7
	2	JDExplore d-team	Vega v2		91.3	90.5	98.6/99.2	99.4	88.2/62.4	94.4/93.9	96.0	77.4	98.6	-0.4	100.0/50.0
+	3	Liam Fedus	ST-MoE-32B		91.2	92.4	96.9/98.0	99.2	89.6/65.8	95.1/94.4	93.5	77.7	96.6	72.3	96.1/94.1
	4	Microsoft Alexander v-team	Turing NLR v5		90.9	92.0	95.9/97.6	98.2	88.4/63.0	96.4/95.9	94.1	77.1	97.3	67.8	93.3/95.5
	5	ERNIE Team - Baidu	ERNIE 3.0		90.6	91.0	98.6/99.2	97.4	88.6/63.2	94.7/94.2	92.6	77.4	97.3	68.6	92.7/94.7
	6	Yi Tay	PaLM 540B		90.4	91.9	94.4/96.0	99.0	88.7/63.6	94.2/93.3	94.1	77.4	95.9	72.9	95.5/90.4
+	7	Zirui Wang	T5 + UDG, Single Model (Google Brain)		90.4	91.4	95.8/97.6	98.0	88.3/63.0	94.2/93.5	93.0	77.9	96.6	69.1	92.7/91.9
+	8	DeBERTa Team - Microsoft	DeBERTa / TuringNLRv4		90.3	90.4	95.7/97.6	98.4	88.2/63.7	94.5/94.1	93.2	77.5	95.9	66.7	93.3/93.8

Wang et al. SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems. NeurIPS 2019.

Great Results With Transformers Outside NLP!

Transformers are not limited to text.

Protein Structure Prediction

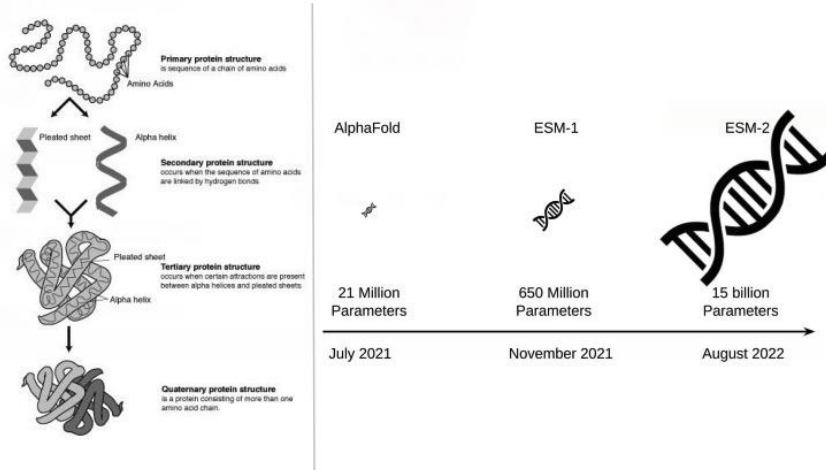
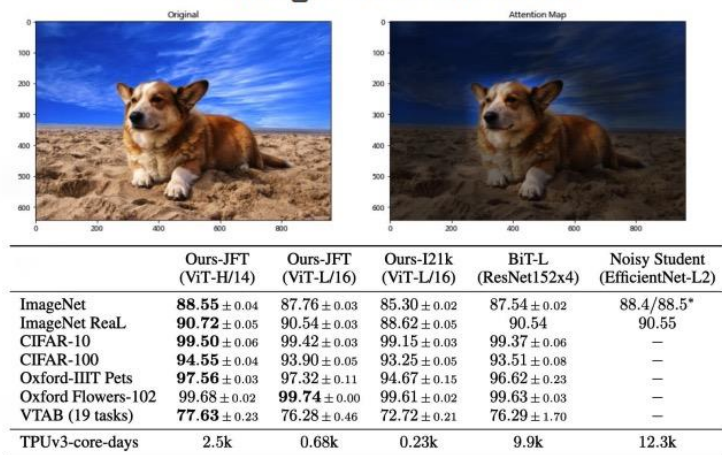


Image Classification



Vision Transformer models pre-trained on JFT-300M dataset outperform ResNet-based baselines on all datasets, while taking substantially less compute to pretrain.

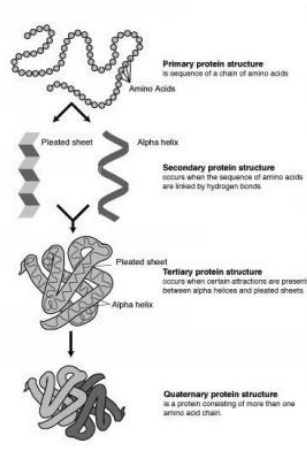
Z. Lin et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. Science 2023.

A. Dosovitskiy et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR 2021

Great Results With Transformers Outside NLP!

Transformers are not limited to text.

Protein Structure Prediction



AlphaFold

ESM-1

ESM-2

21 Million
Parameters

650 Million
Parameters

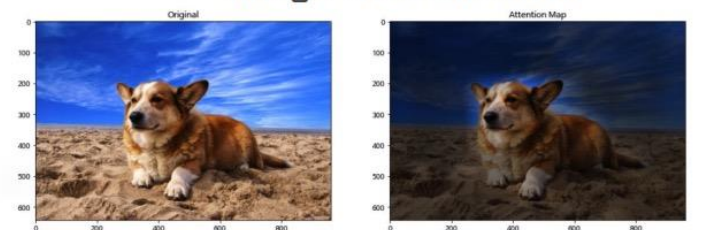
15 billion
Parameters

July 2021

November 2021

August 2022

Image Classification



	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet Real	90.72 ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	99.50 ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	—
CIFAR-100	94.55 ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	—
Oxford-IIIT Pets	97.56 ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	—
Oxford Flowers-102	99.68 ± 0.02	99.74 ± 0.00	99.61 ± 0.02	99.63 ± 0.03	—
VTAB (19 tasks)	77.63 ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	—
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

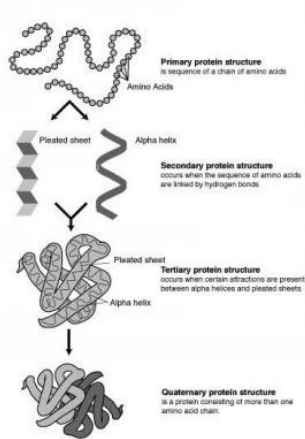
Vision Transformer models pre-trained on JFT-300M dataset outperform ResNet-based baselines on all datasets, while taking substantially less compute to pretrain.

Q: Why can the same model work for text and images?

Great Results With Transformers Outside NLP!

Transformers are not limited to text.

Protein Structure Prediction



AlphaFold

ESM-1

ESM-2

21 Million
Parameters

650 Million
Parameters

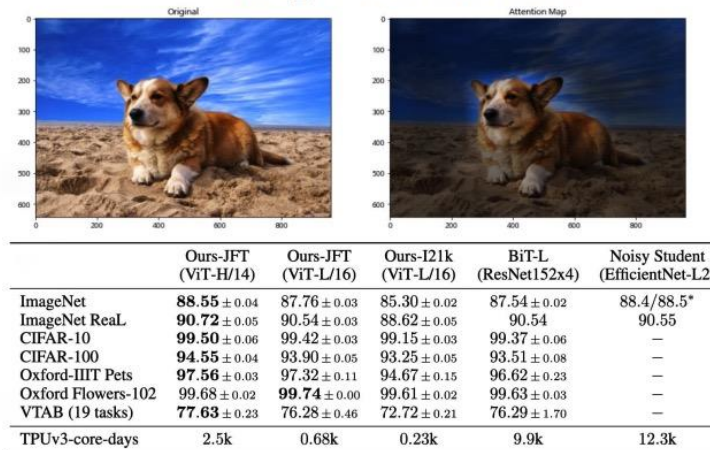
15 billion
Parameters

July 2021

November 2021

August 2022

Image Classification



Vision Transformer models pre-trained on JFT-300M dataset outperform ResNet-based baselines on all datasets, while taking substantially less compute to pretrain.

Q: Why can the same model work for text and images?

A: Input = sequence of tokens (words or image patches).

Motivation for Transformer

- Minimise (or at least not increase) computational complexity per layer.
Computational efficiency
- Minimise path length between any pair of words to facilitate learning of long-range dependencies.
Shorter interaction distance
- Maximise the amount of computation that can be parallelised.
Parallel processing

Q: Which matters most for **large models** with billions of parameters?

Motivation for Transformer

- Minimise (or at least not increase) computational complexity per layer.
Computational efficiency
- Minimise path length between any pair of words to facilitate learning of long-range dependencies.
Shorter interaction distance
- Maximise the amount of computation that can be parallelised.
Parallel processing

Q: Which matters most for **large models** with billions of parameters?

A: Parallelisability.

Transformer Motivation: Computational Complexity Per Layer

When sequence length (n) $\ll$ representation dimension (d), the complexity per layer is lower for a Transformer model compared to RNN models.

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Table 1 of paper by A. Vaswani et al. Attention Is All You Need. NeurIPS 2017.

When $n < d$, Transformers are more efficient!

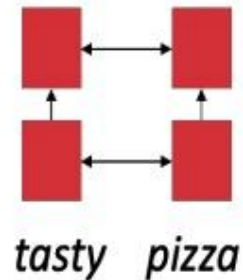
Transformer Motivation: Minimise Linear Interaction Distance

- RNN is unrolled “left-to-right”.
 - It encodes linear locality: a useful heuristic!
 - Problem: RNN takes $O(\text{sequence length})$ steps for distant word pairs to interact.
-
- Hard to learn long-distance dependencies due to gradient problems.
 - Linear order of words is “baked in”; we already know sequential structure doesn't tell the whole story...

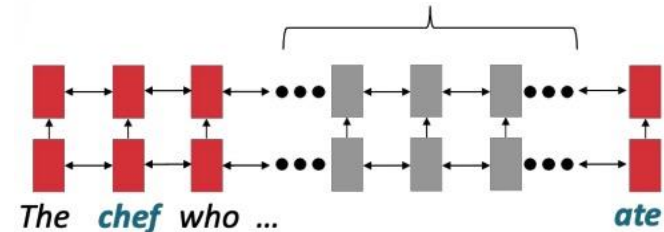
RNN must pass information step-by-step!

Transformer allows all tokens to interact directly.

Nearby words often affect each other's meanings



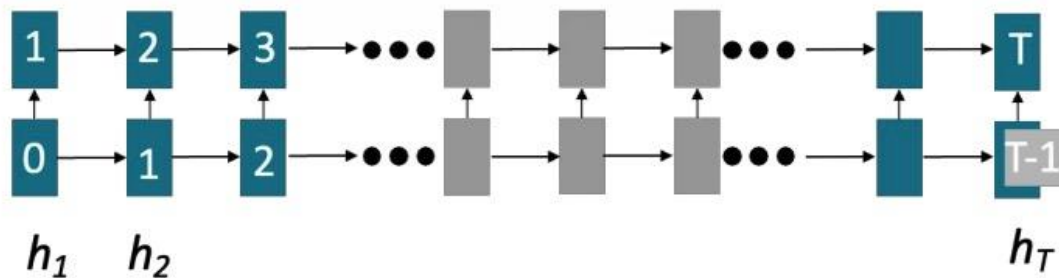
Info about chef has gone through $O(\text{sequence length})$ many layers!



Transformer Motivation: Maximise Parallelisability

Forward and backward passes have $O(\text{sequence length})$ unparallelizable operations.

- GPUs (and TPUs) can perform many independent computations at once!
- But future RNN hidden states can't be computed in full before past RNN hidden states have been computed.
- Inhibits training on very large datasets!
- Particularly problematic as sequence length increases, as we can no longer batch many examples together due to memory limitations.



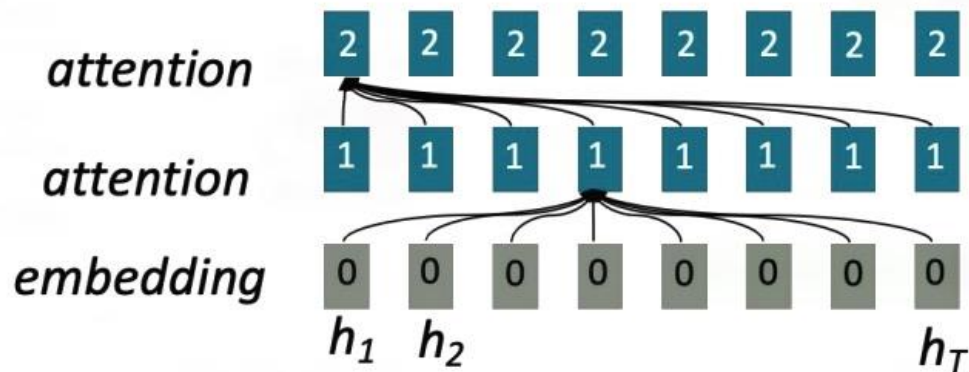
RNN cannot compute future states before previous ones.

Transformer computes all token representations simultaneously.

Numbers indicate min # of steps before a state can be computed

High-Level Architecture: Transformer is all about (Self) Attention

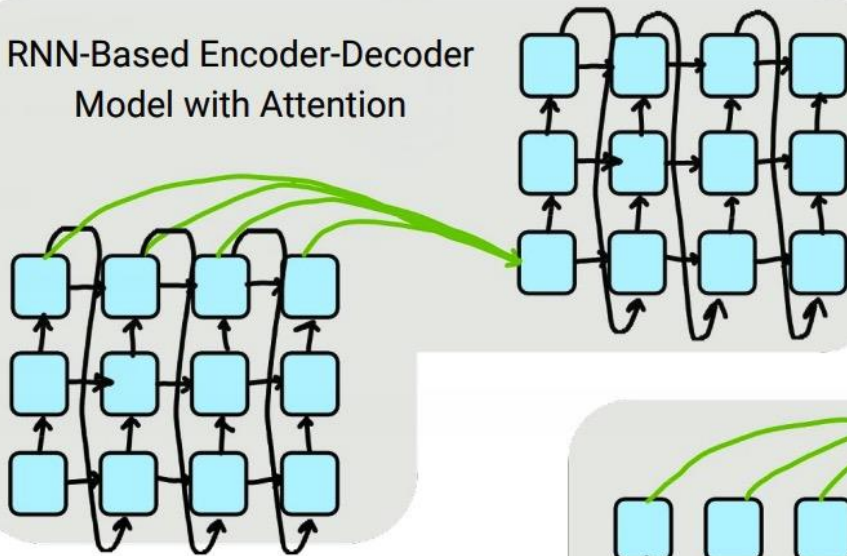
- Self-attention is encoder-encoder (or decoder-decoder) attention where each word attends to each other word within the input (or output).



All words attend to all words in previous layer; most arrows here are omitted.

Computational Dependencies for Recurrence vs. Attention

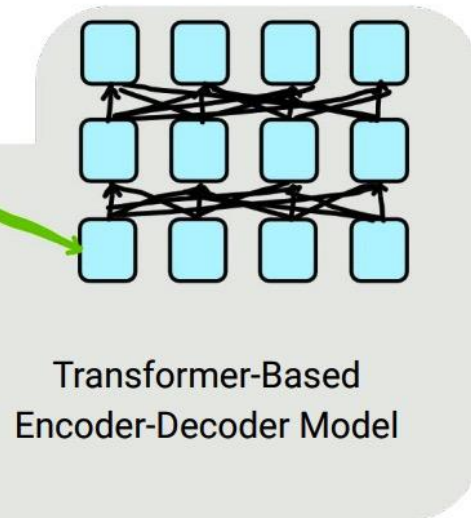
RNN-Based Encoder-Decoder Model with Attention



Max interaction distance = 1
in Transformer vs **n** in RNN.

Transformer Advantages:

- # unparallelizable operations does not increase with sequence length.
- Each word interacts with each other, so maximum interaction distance is $O(1)$.



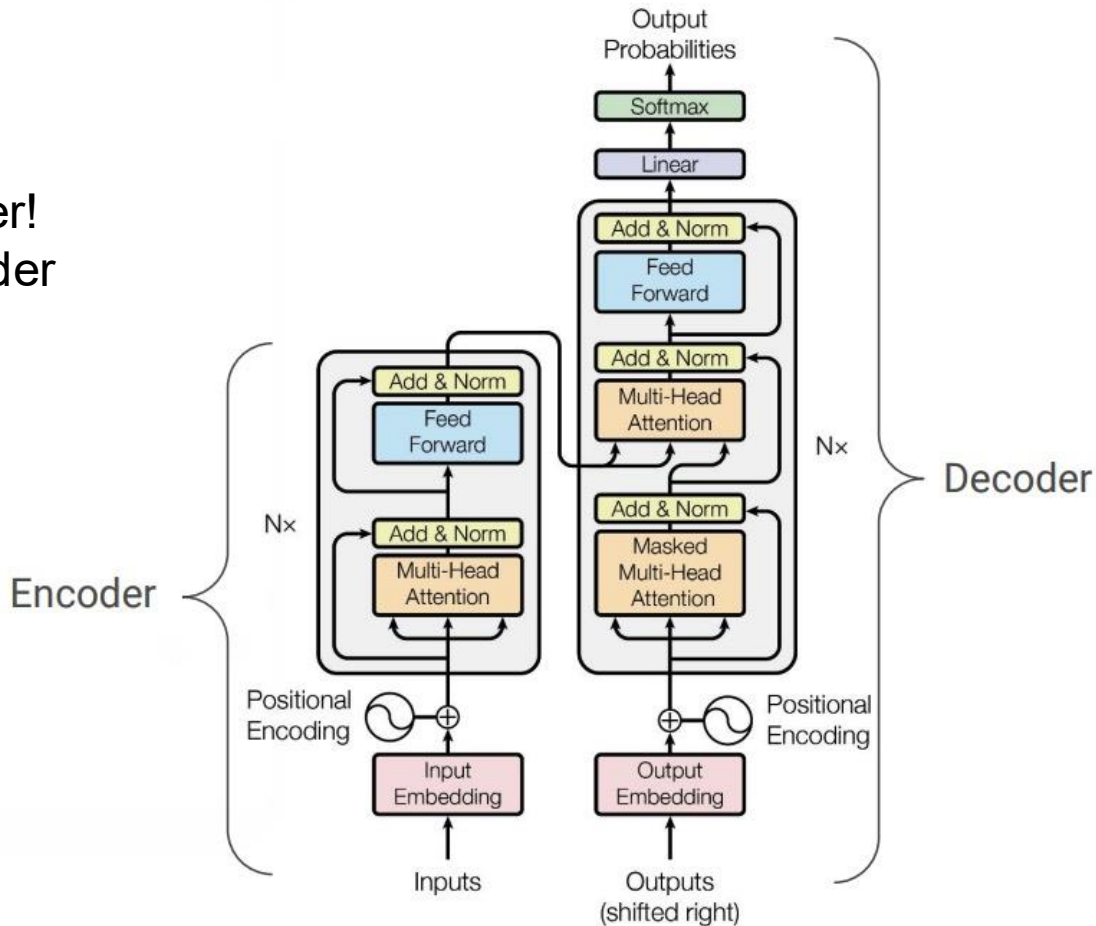
The Transformer Encoder-Decoder

Next, we will learn exactly how the Transformer architecture works:

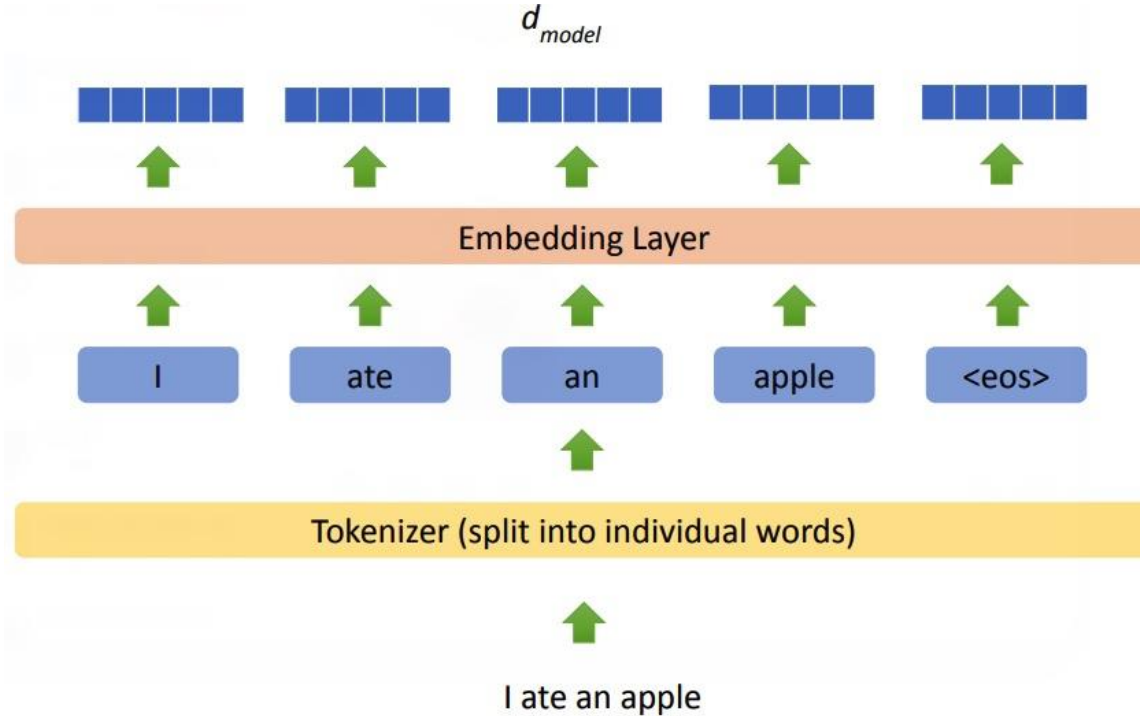
- First, we will talk about the Encoder!
- Next, we will go through the Decoder (which is quite similar)!

Encoder processes input sentence.

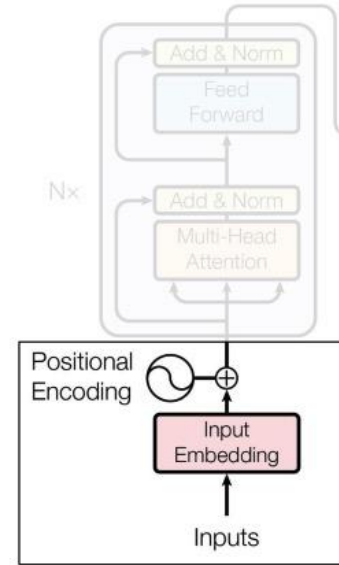
Decoder predicts output step-by-step with masked attention.



First step: Input Embeddings



Generate Input Embeddings



Position Encoding

For each position, an embedded input is moved the same distance but at a different angle. Inputs that are close to each other in the sequence have similar perturbations, but inputs that are far apart are perturbed in different directions.

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

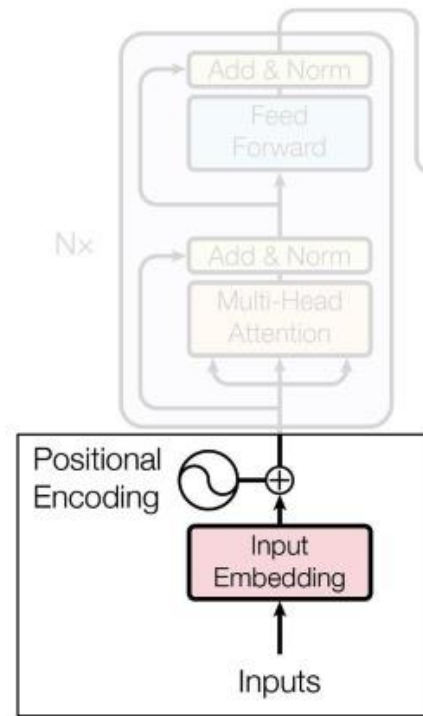
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

pos -> idx of the token in input sentence

i -> ith dimension out of d

d_model -> embedding dimension of each token

Different calculations for odd and even embedding indices



Q: What if we removed positional encoding?

Position Encoding

For each position, an embedded input is moved the same distance but at a different angle. Inputs that are close to each other in the sequence have similar perturbations, but inputs that are far apart are perturbed in different directions.

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

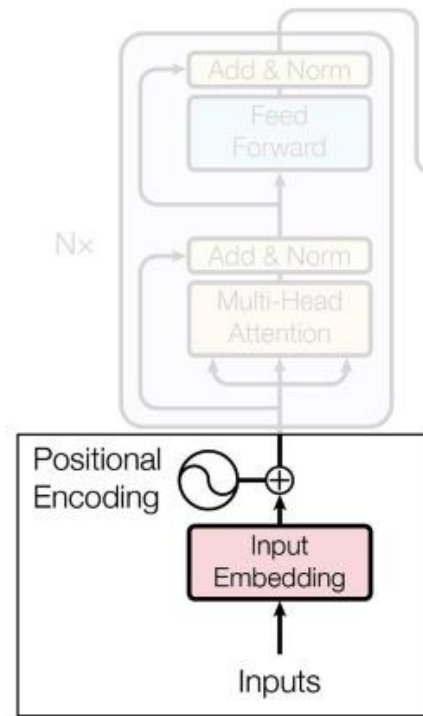
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

pos -> idx of the token in input sentence

i -> ith dimension out of d

d_model -> embedding dimension of each token

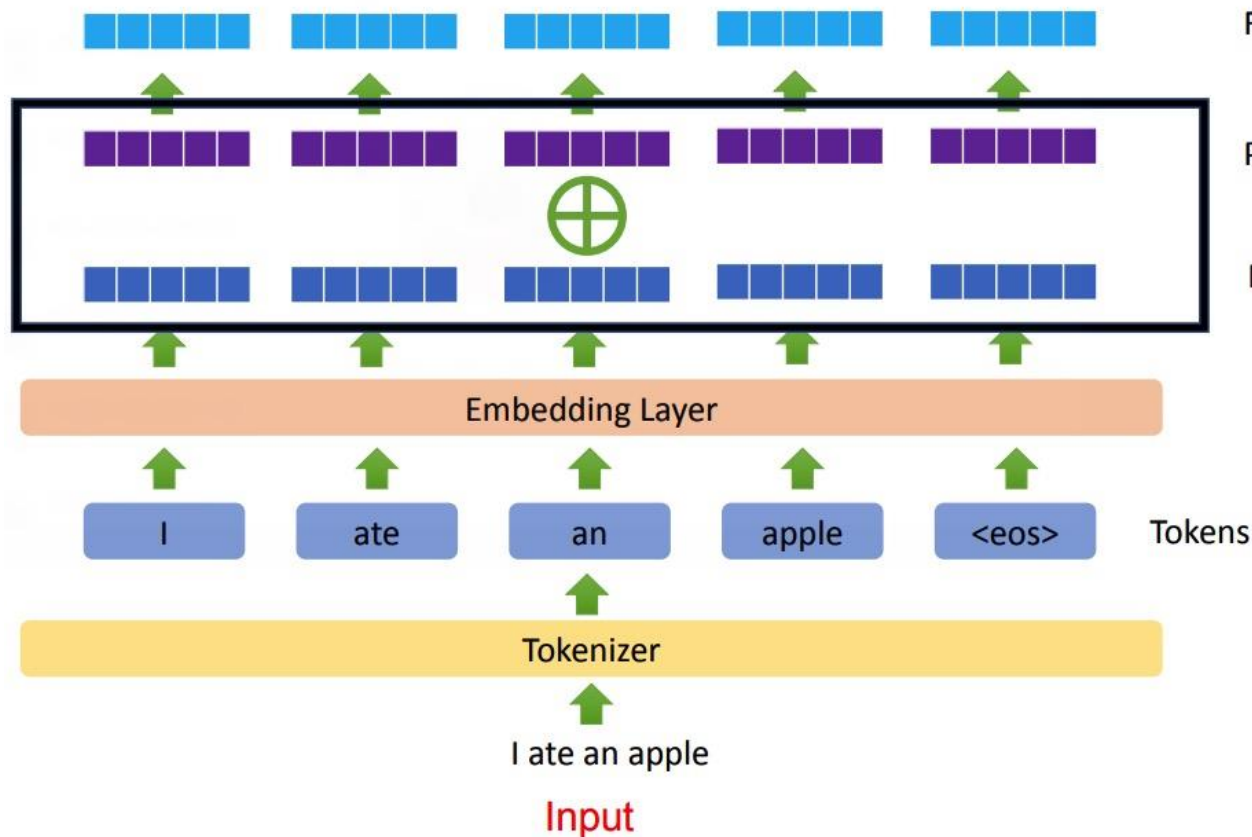
Different calculations for odd and even embedding indices



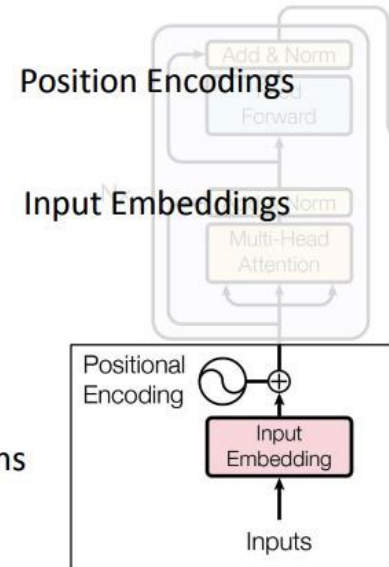
Q: What if we removed positional encoding?

A: Model loses word order understanding.

Final Input Embeddings



Final Input Embeddings

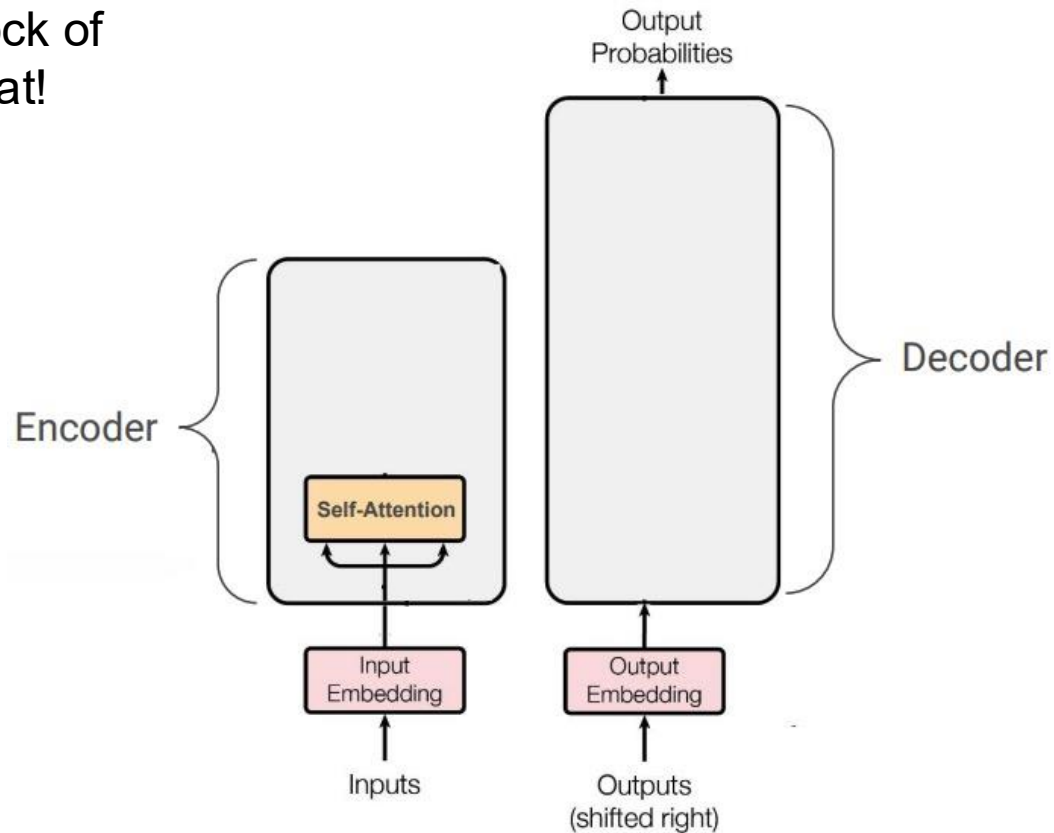


Tokens

Encoder: Self-Attention

Self-Attention is the core building block of Transformer, so let's first focus on that!

Q: Does attention depend on other sentences?

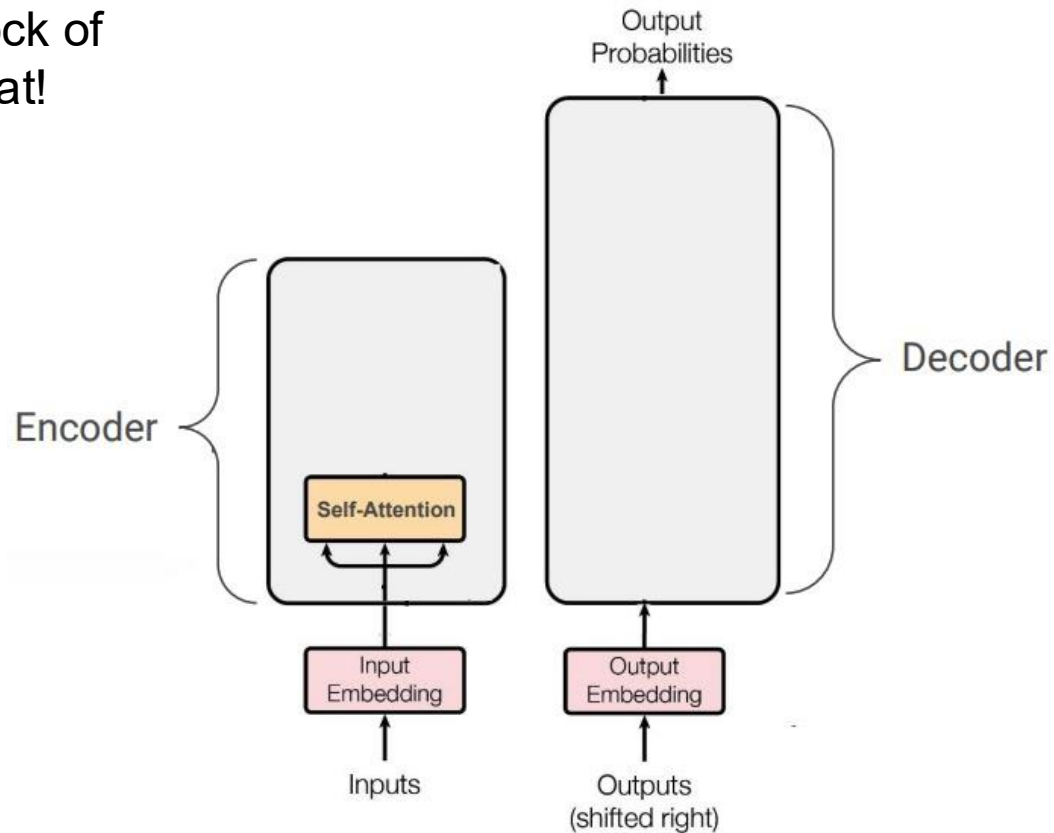


Encoder: Self-Attention

Self-Attention is the core building block of Transformer, so let's first focus on that!

Q: Does attention depend on other sentences?

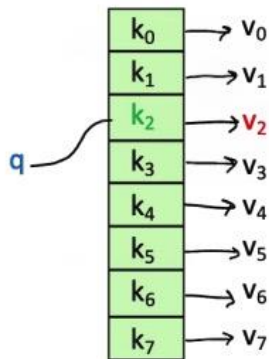
A: No, only the current sequence.



Intuition for Attention Mechanism

Let's think of attention as a "fuzzy" or approximate hashtable. To look up a value, we compare a query against keys in a table.

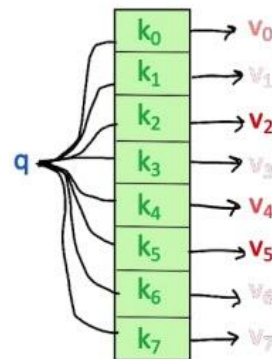
In a hashtable:



Query = what I'm looking for
Key = what I represent
Value = information I provide

Each query (hash) maps to exactly one key-value pair.

In (self-)attention:



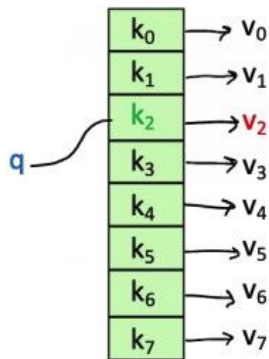
Q: Which vector determines importance?

Each query matches each key to varying degrees. We return a sum of values **weighted by the query-key** match.

Intuition for Attention Mechanism

Let's think of attention as a "fuzzy" or approximate hashtable. To look up a value, we compare a query against keys in a table.

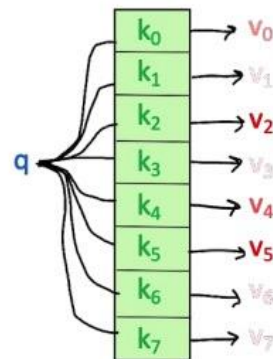
In a hashtable:



Query = what I'm looking for
Key = what I represent
Value = information I provide

Each query (hash) maps to exactly one key-value pair.

In (self-)attention:



Q: Which vector determines importance?

A: Query-key similarity.

Each query matches each key to varying degrees. We return a sum of values **weighted by the query-key** match.

Self-Attention

Step 1: For each word x_i , calculate its **query**, **key**, and **value**.

$$q_i = W^Q x_i \quad k_i = W^K x_i \quad v_i = W^V x_i$$

Step 2: Calculate attention score between **query** and **keys**.

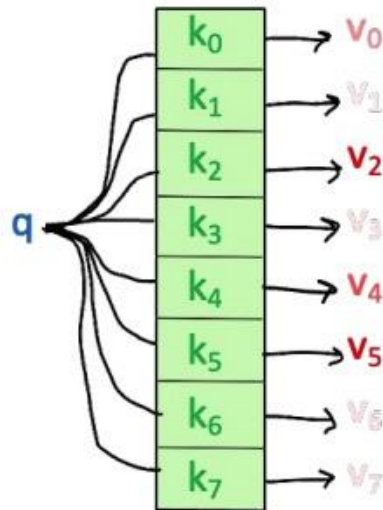
$$e_{ij} = q_i \cdot k_j$$

Step 3: Take the softmax to normalise attention scores.

$$\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}$$

Step 4: Take a weighted sum of **values**.

$$\text{Output}_i = \sum_j \alpha_{ij} v_j$$



Self-Attention

Step 1: With embeddings stacked in X , calculate **queries**, **keys**, and **values**.

$$Q = XW^Q \quad K = XW^K \quad V = XW^V$$

Step 2: Calculate attention score between **query** and **keys**.

$$E = QK^T$$

Step 3: Take the softmax to normalise attention scores.

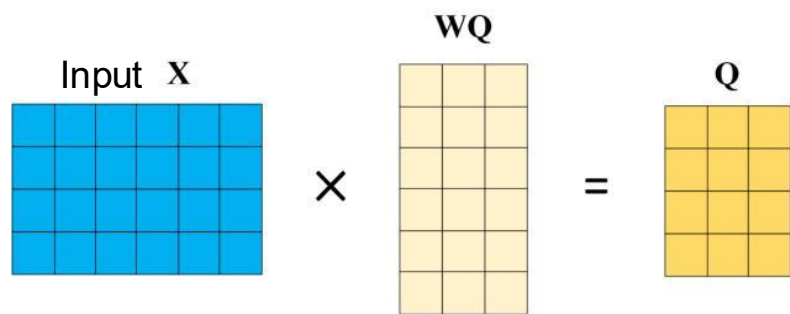
$$A = \text{softmax}(E)$$

Step 4: Take a weighted sum of **values**.

$$\text{Output} = AV$$

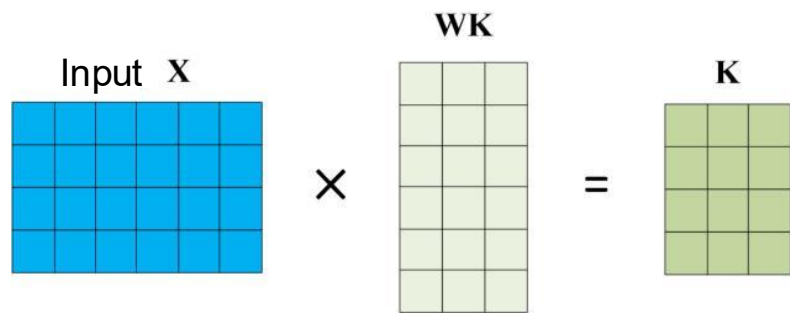
$$\text{Output} = \text{softmax}(QK^T)V$$

Input X $\times$ WQ $=$ Q



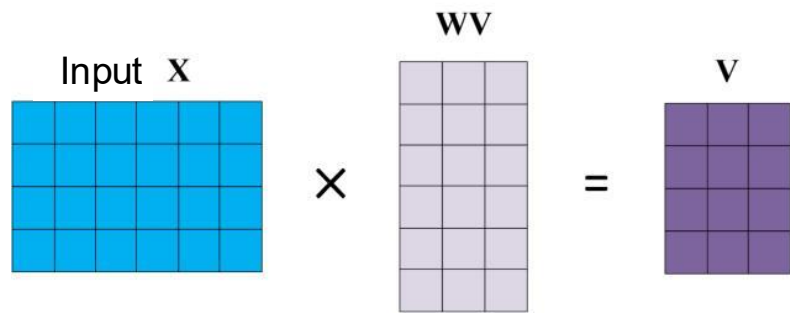
The diagram illustrates the matrix multiplication of Input X (a 4x8 blue grid) by WQ (an 8x4 yellow grid) to produce the output Q (a 4x4 yellow grid). The multiplication is represented by the equation $X \times WQ = Q$.

Input X $\times$ WK $=$ K

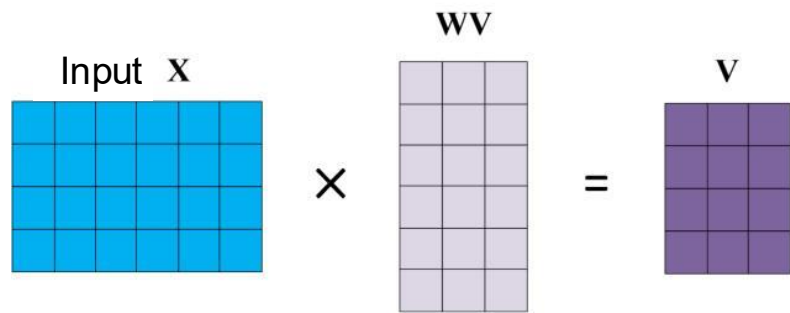
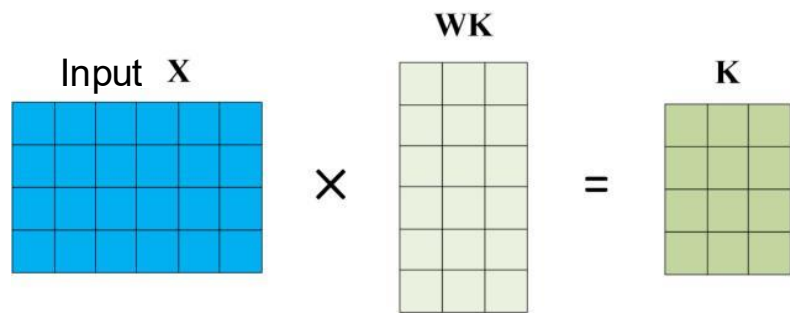
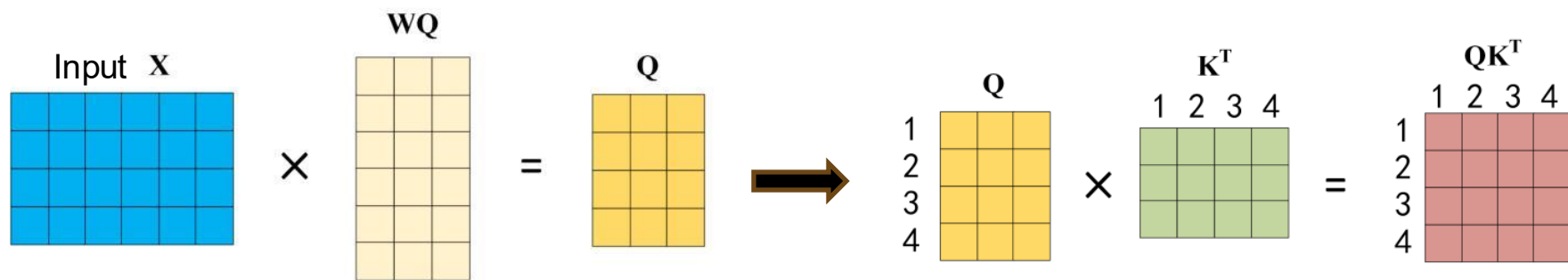


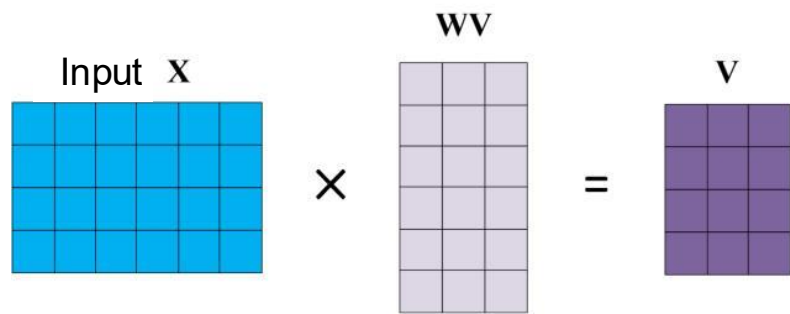
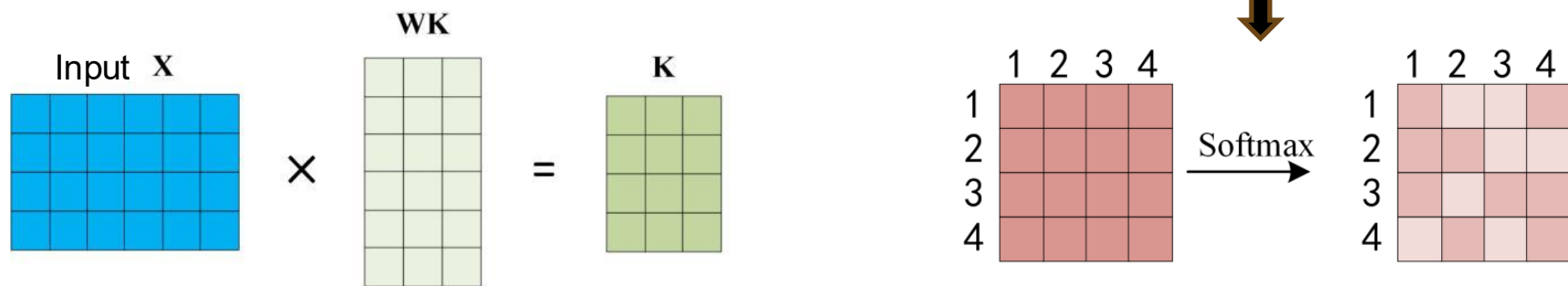
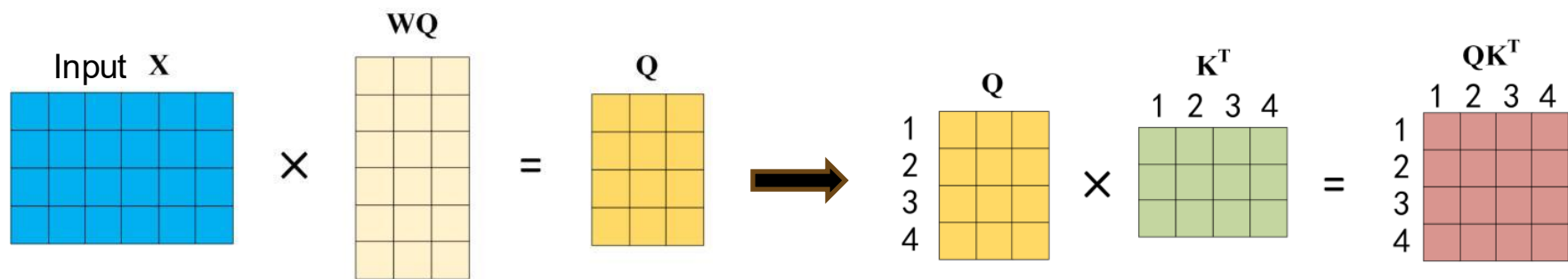
The diagram illustrates the matrix multiplication of Input X (a 4x8 blue grid) by WK (an 8x4 light green grid) to produce the output K (a 4x4 light green grid). The multiplication is represented by the equation $X \times WK = K$.

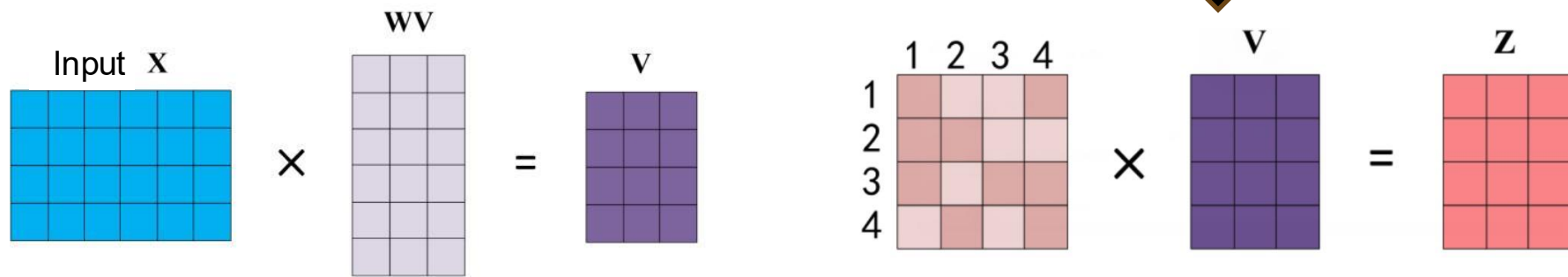
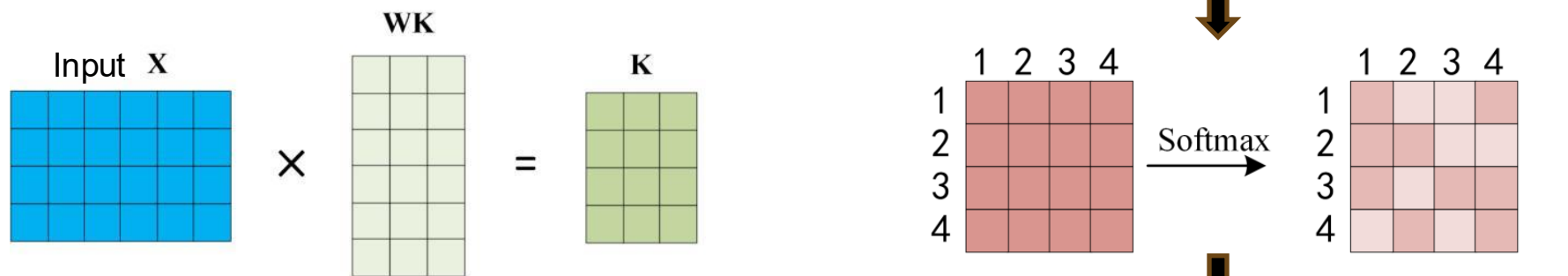
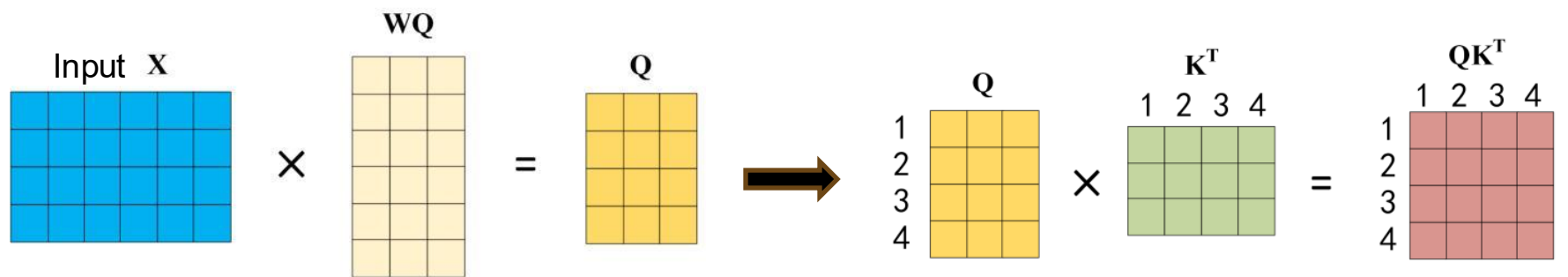
Input X $\times$ WV $=$ V



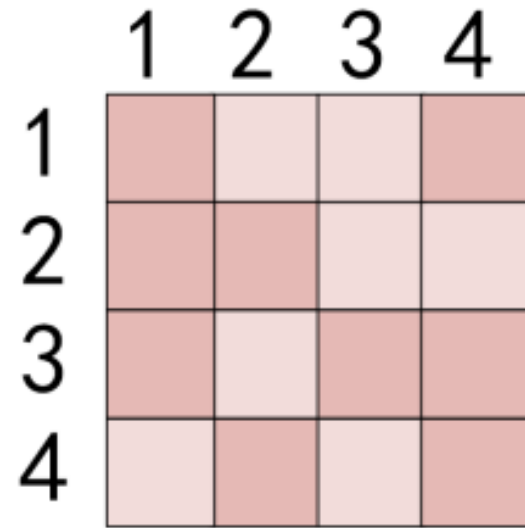
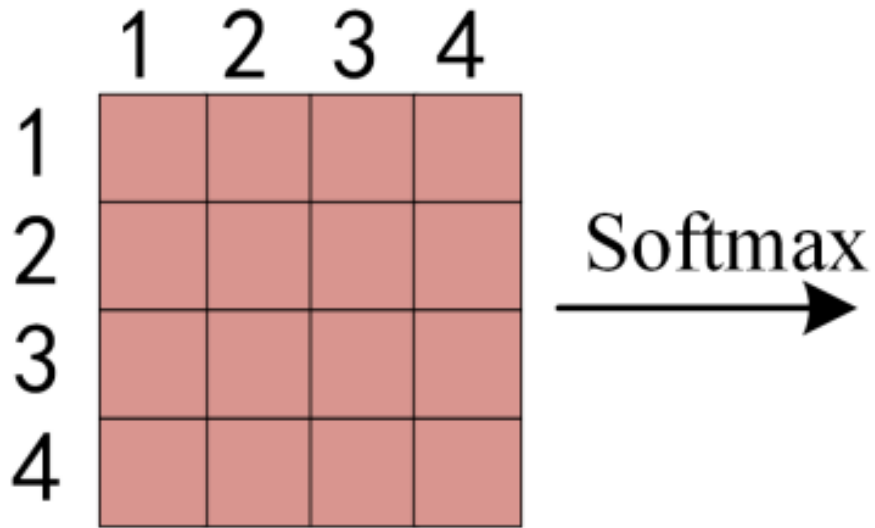
The diagram illustrates the matrix multiplication of Input X (a 4x8 blue grid) by WV (an 8x4 light purple grid) to produce the output V (a 4x4 dark purple grid). The multiplication is represented by the equation $X \times WV = V$.







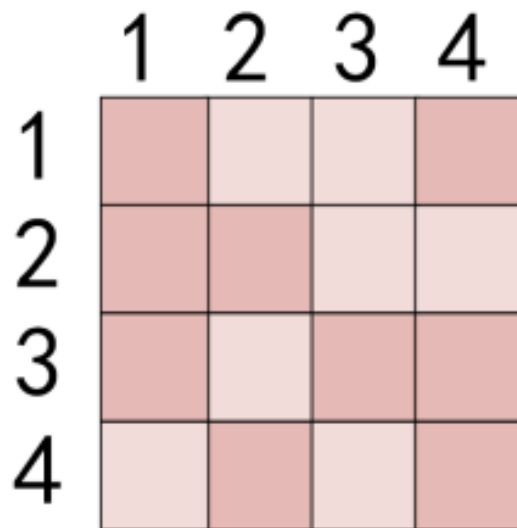
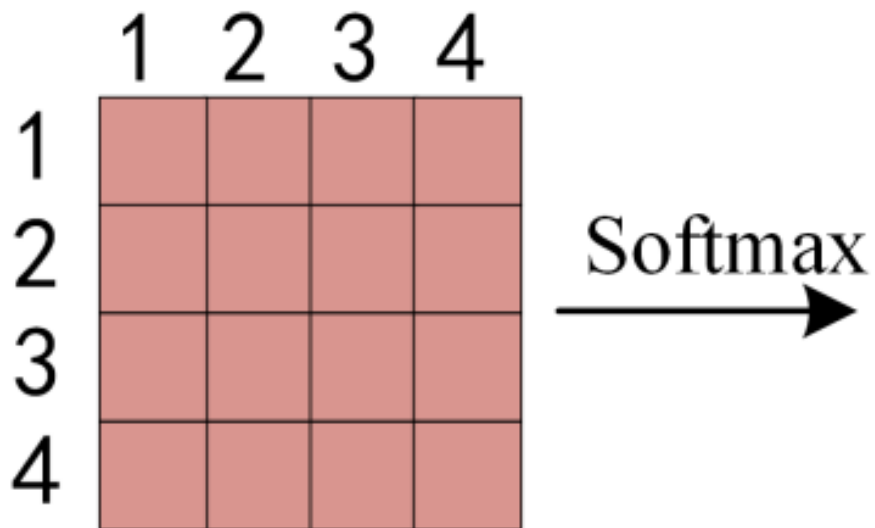
Self-Attention



Attention coefficient matrix

Q1: Why use softmax?

Self-Attention



Attention coefficient matrix

Q1: Why use softmax?

A: Convert scores to probabilities.

Self-Attention

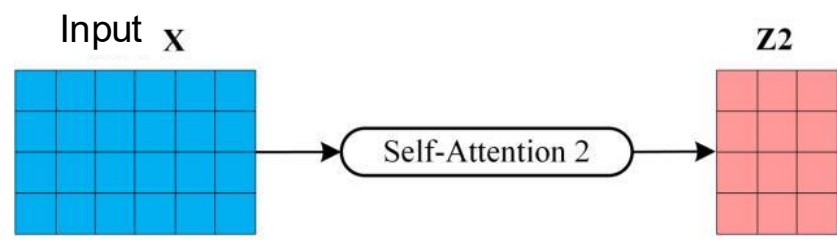
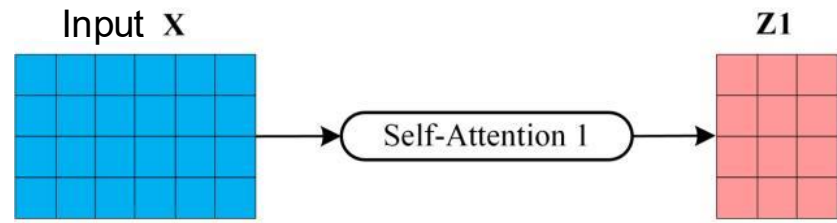
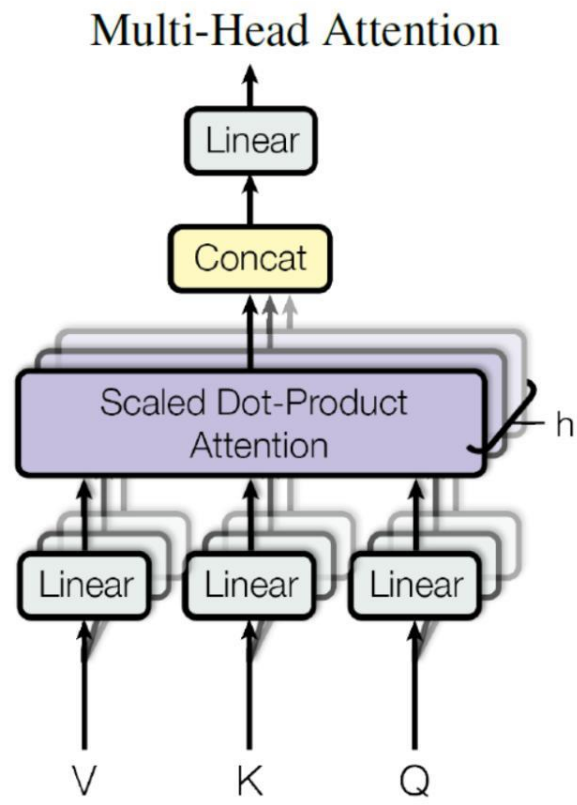
$$\begin{array}{c} 1 & 2 & 3 & 4 \\ \begin{array}{c} 1 \\ 2 \\ 3 \\ 4 \end{array} & \begin{array}{|c|c|c|c|} \hline \text{light red} & \text{light red} & \text{light red} & \text{light red} \\ \hline \text{light red} & \text{light red} & \text{light red} & \text{light red} \\ \hline \text{light red} & \text{light red} & \text{light red} & \text{light red} \\ \hline \text{light red} & \text{light red} & \text{light red} & \text{light red} \\ \hline \end{array} & \times & \begin{array}{|c|c|c|} \hline \text{purple} & \text{purple} & \text{purple} \\ \hline \text{purple} & \text{purple} & \text{purple} \\ \hline \text{purple} & \text{purple} & \text{purple} \\ \hline \text{purple} & \text{purple} & \text{purple} \\ \hline \end{array} & = & \begin{array}{|c|c|c|} \hline \text{red} & \text{red} & \text{red} \\ \hline \text{red} & \text{red} & \text{red} \\ \hline \text{red} & \text{red} & \text{red} \\ \hline \text{red} & \text{red} & \text{red} \\ \hline \end{array} \\
 \end{array}$$

$$\begin{array}{c} \mathbf{Z}_1 \\ \begin{array}{|c|c|c|} \hline \text{red} & \text{red} & \text{red} \\ \hline \end{array} \end{array} = 1 \begin{array}{c} 1 & 2 & 3 & 4 \\ \begin{array}{|c|c|c|c|} \hline 0.3 & 0.2 & 0.2 & 0.3 \\ \hline \end{array} \end{array} \times \begin{array}{c} \mathbf{V} \\ \begin{array}{|c|c|c|} \hline \text{purple} & \text{purple} & \text{purple} \\ \hline \text{purple} & \text{purple} & \text{purple} \\ \hline \text{purple} & \text{purple} & \text{purple} \\ \hline \text{purple} & \text{purple} & \text{purple} \\ \hline \end{array} \end{array}$$

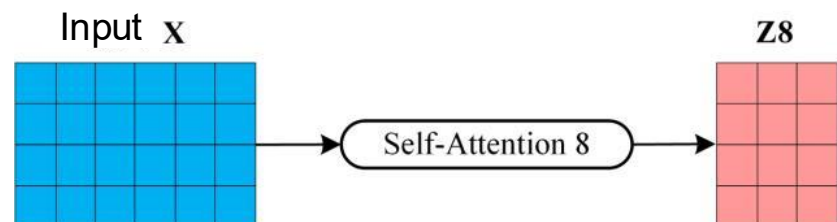
$$= 0.3 \times \begin{array}{|c|c|c|} \hline \text{purple} & \text{purple} & \text{purple} \\ \hline \end{array} + 0.2 \times \begin{array}{|c|c|c|} \hline \text{purple} & \text{purple} & \text{purple} \\ \hline \end{array} + 0.2 \times \begin{array}{|c|c|c|} \hline \text{purple} & \text{purple} & \text{purple} \\ \hline \end{array} + 0.3 \times \begin{array}{|c|c|c|} \hline \text{purple} & \text{purple} & \text{purple} \\ \hline \end{array}$$

- The first row of the Softmax matrix in the figure above represents the attention weights between word 1 and all other words.
- The final output Z_1 for word 1 is obtained by taking the values V_i of all words and combining them according to the proportions given by the attention weights

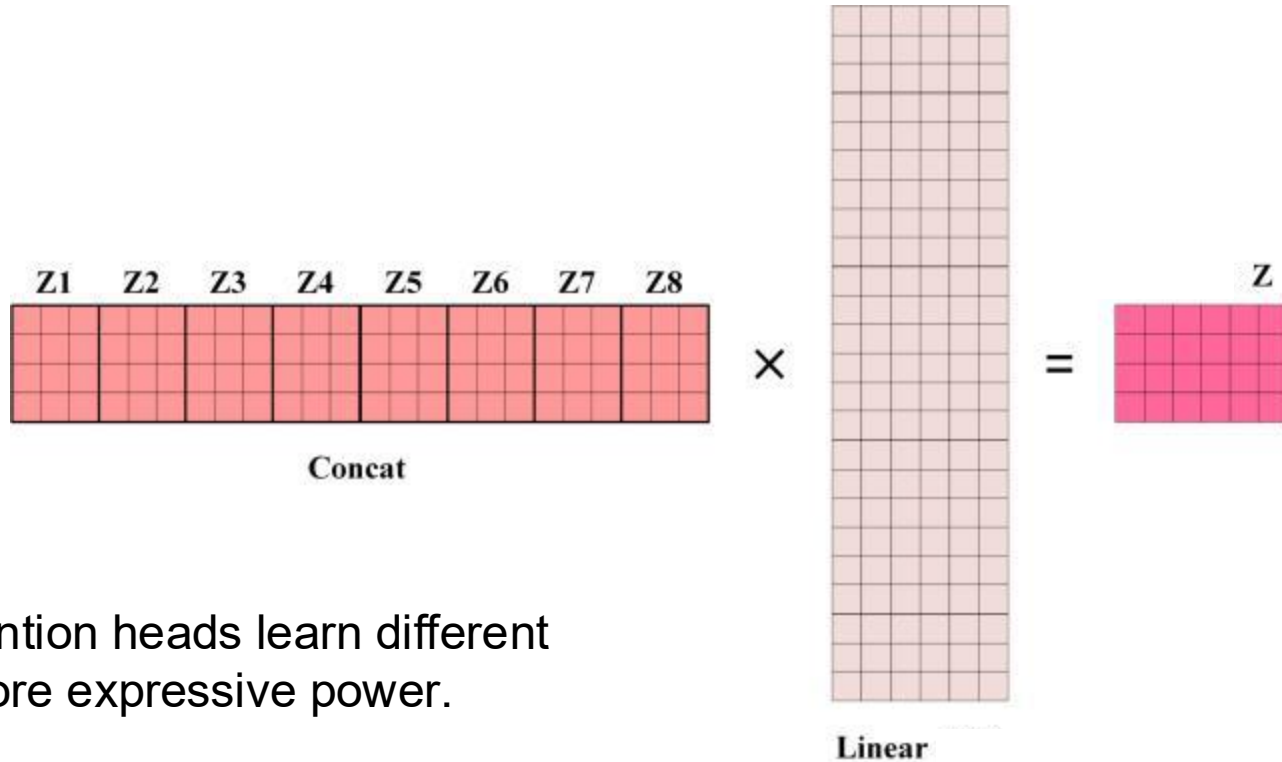
Multi-Head Attention



.....



Multi-Head Attention

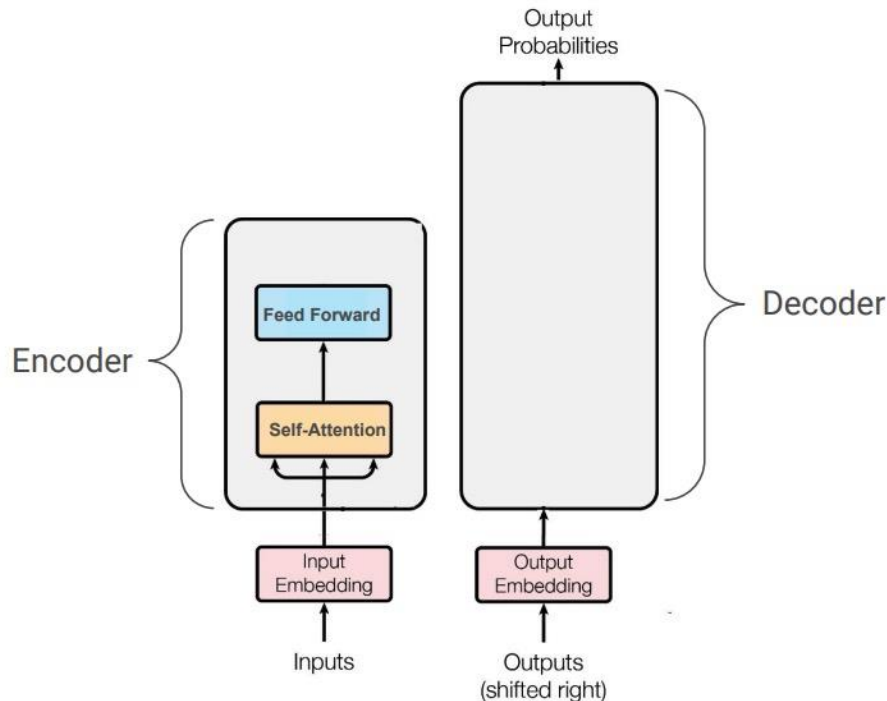
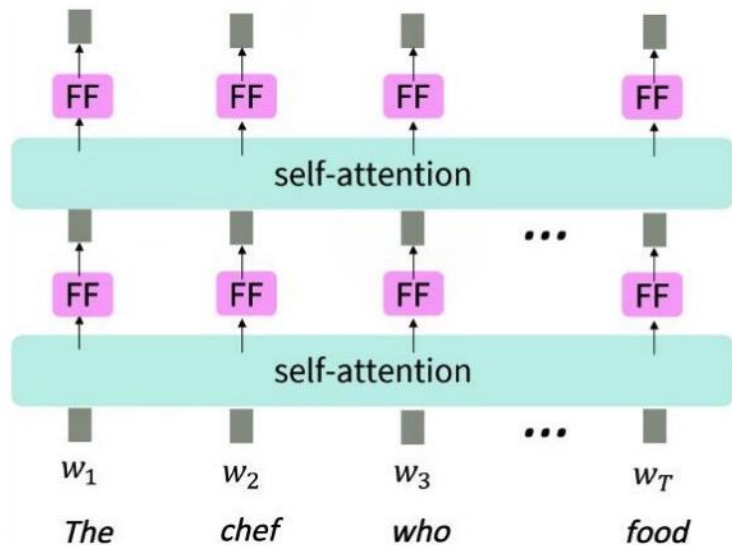


Multiple attention heads learn different relations. More expressive power.

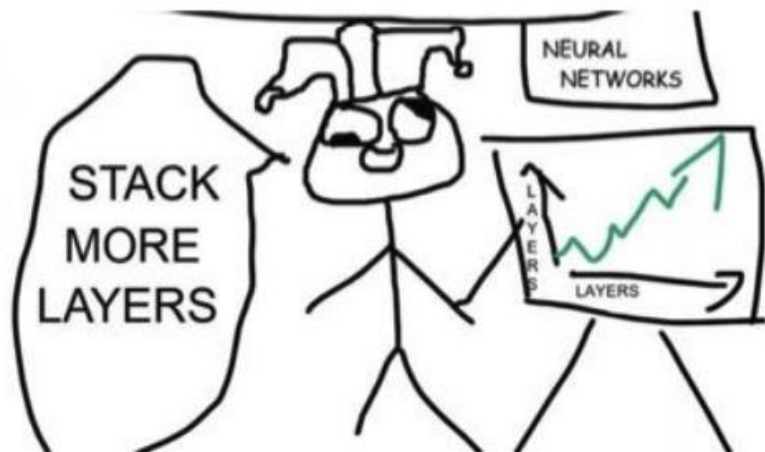
Attention Isn't All You Need!

Equation for Feed Forward Layer

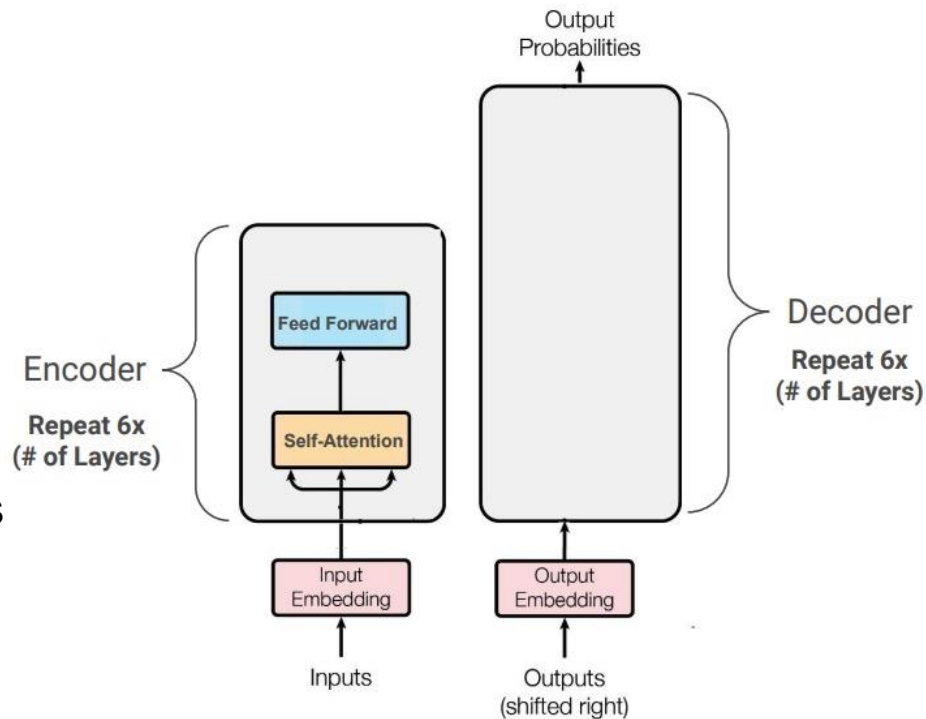
$$\begin{aligned} m_i &= \text{MLP}(\text{output}_i) \\ &= W_2 * \text{ReLU}(W_1 \times \text{output}_i + b_1) + b_2 \end{aligned}$$



Making This Work For Deep Networks



- Training Trick #1: Residual Connections
- Training Trick #2: LayerNorm
- Training Trick #3: Scaled Dot Product Attention



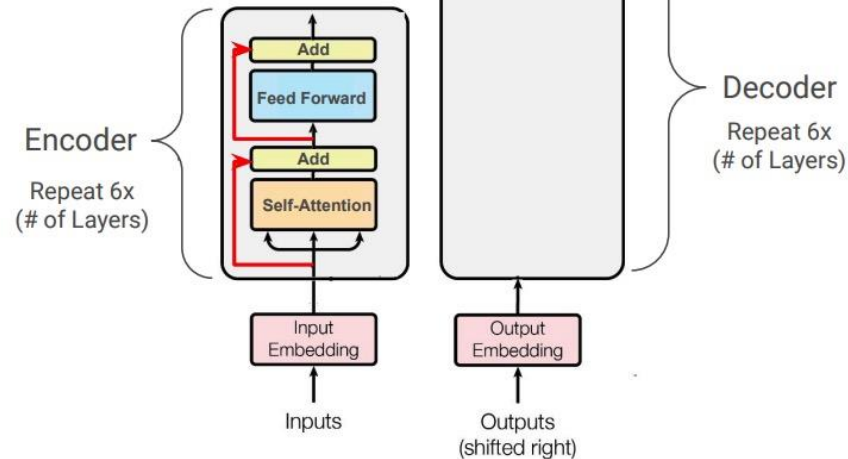
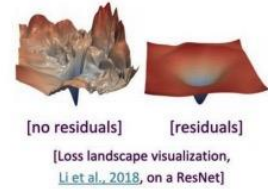
Training Trick 1: Residual Connections

- Residual connections are a simple but powerful technique from computer vision.
- Deep networks are surprisingly bad at learning the identity function!
- Therefore, directly passing "raw" embeddings to the next layer can actually be very helpful!

$$x_\ell = F(x_{\ell-1}) + x_{\ell-1}$$

- This prevents the network from "forgetting" or distorting important information as it is processed by many layers.

Residual connections are also thought to smooth the loss landscape and make training easier!



Q: Similar idea used where else?

Training Trick 1: Residual Connections

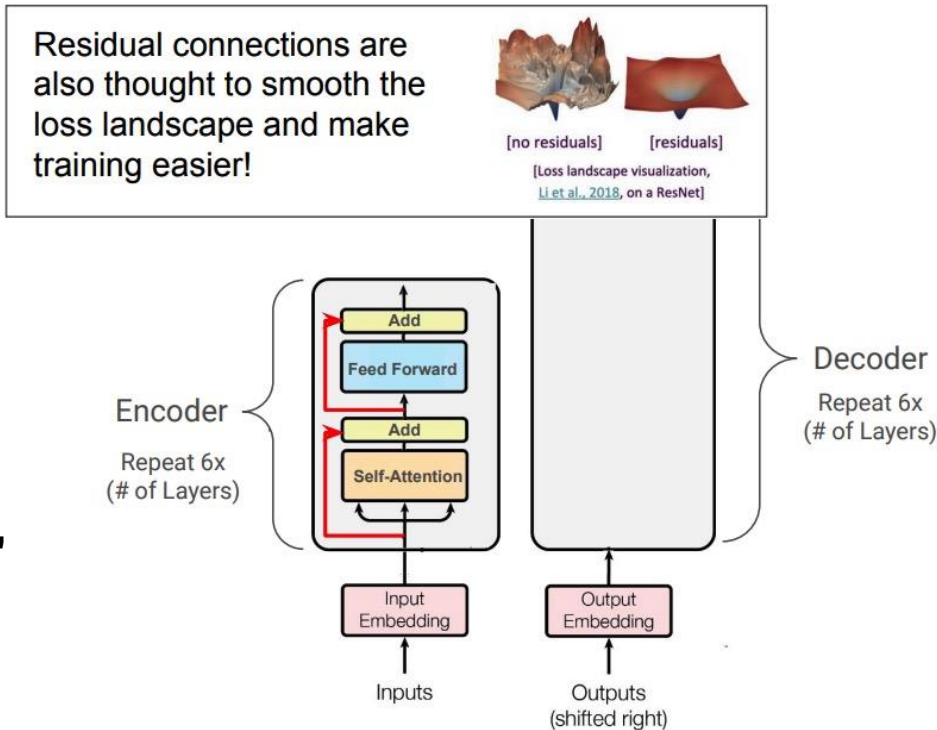
- Residual connections are a simple but powerful technique from computer vision.
- Deep networks are surprisingly bad at learning the identity function!
- Therefore, directly passing "raw" embeddings to the next layer can actually be very helpful!

$$x_{\ell} = F(x_{\ell-1}) + x_{\ell-1}$$

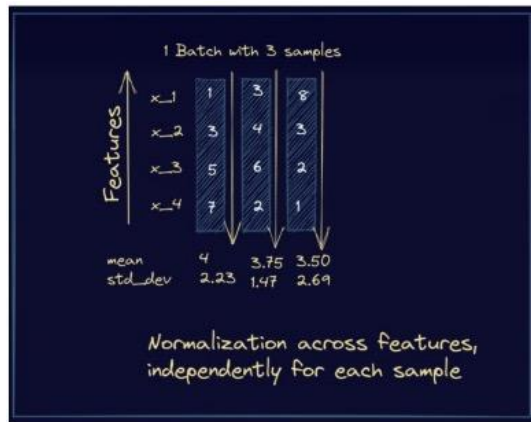
- This prevents the network from "forgetting" or distorting important information as it is processed by many layers.

Q: Similar idea used where else?

A: ResNet.



Training Trick 2: Layer Normalisation



An Example of How LayerNorm Works
(Image by Bala Priya C, Pinecone)

Mean:

$$\mu^l = \frac{1}{H} \sum_{i=1}^H a_i^l$$

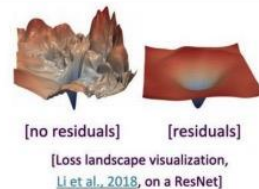
Standard Deviation:

$$\sigma^l = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_i^l - \mu^l)^2}$$

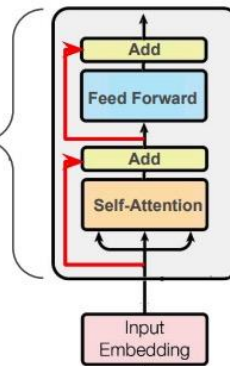
$$x^{\ell'} = \frac{x^{\ell} - \mu^{\ell}}{\sigma^{\ell} + \epsilon}$$

Q: Difference vs BatchNorm?

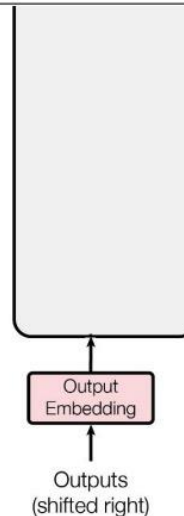
Residual connections are also thought to smooth the loss landscape and make training easier!



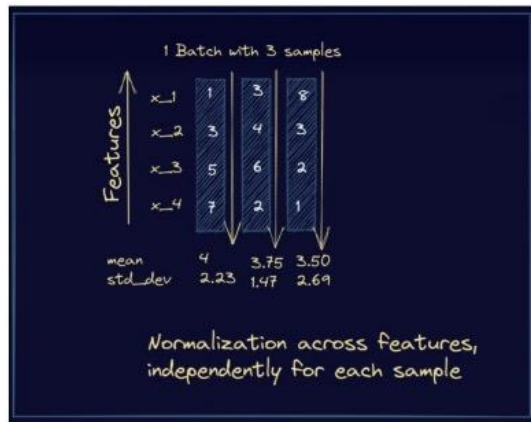
Encoder
Repeat 6x
(# of Layers)



Decoder
Repeat 6x
(# of Layers)



Training Trick 2: Layer Normalisation



An Example of How LayerNorm Works
(Image by Bala Priya C, Pinecone)

Mean:

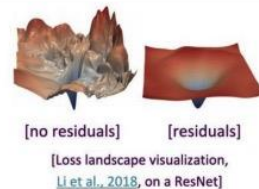
$$\mu^l = \frac{1}{H} \sum_{i=1}^H a_i^l$$

Standard Deviation:

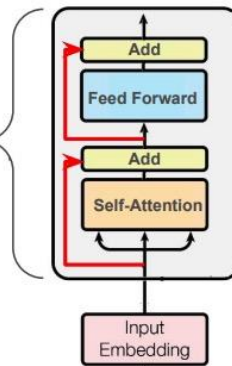
$$\sigma^l = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_i^l - \mu^l)^2}$$

$$x^{\ell'} = \frac{x^{\ell} - \mu^{\ell}}{\sigma^{\ell} + \epsilon}$$

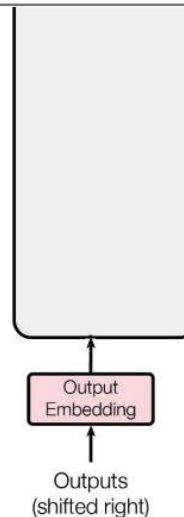
Residual connections are also thought to smooth the loss landscape and make training easier!



Encoder
Repeat 6x
(# of Layers)



Decoder
Repeat 6x
(# of Layers)



Q: Difference vs BatchNorm?

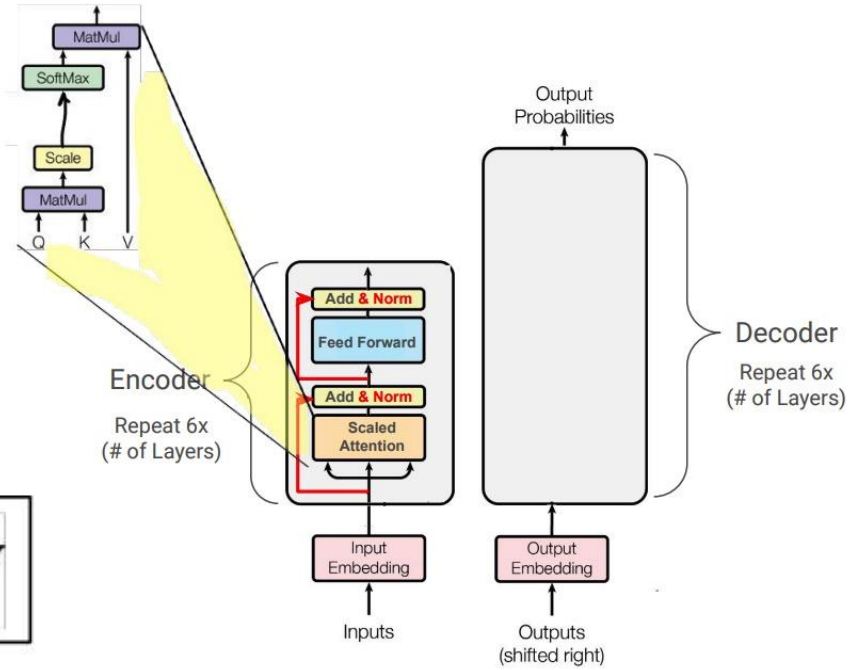
A: BatchNorm depends on batch, LayerNorm doesn't.

Training Trick 3: Scaled Dot Product Attention

- After LayerNorm, the mean and variance of vector elements is 0 and 1, respectively.
- However, the dot product still tends to take on extreme values, as its variance scales with dimensionality d_k .

Updated Self-Attention Equation:

$$\text{Output} = \text{softmax}\left(QK^T / \sqrt{d_k}\right)V$$



**Next: Decoder, Large
Models**